FLUTTER COURSE · PRACTICE PROJECT FOR SESSIONS 1–4

Paws & Homes

A pet adoption app. People browse pets waiting for a home, open a pet's profile, and fill in an adoption form. You will build it from scratch using only what we have covered so far: Dart, widgets, layouts, theming, navigation and forms.

WHAT YOU HAND IN

A GitHub repository with the working app, a README with screenshots of all four screens, and at least 5 commits.

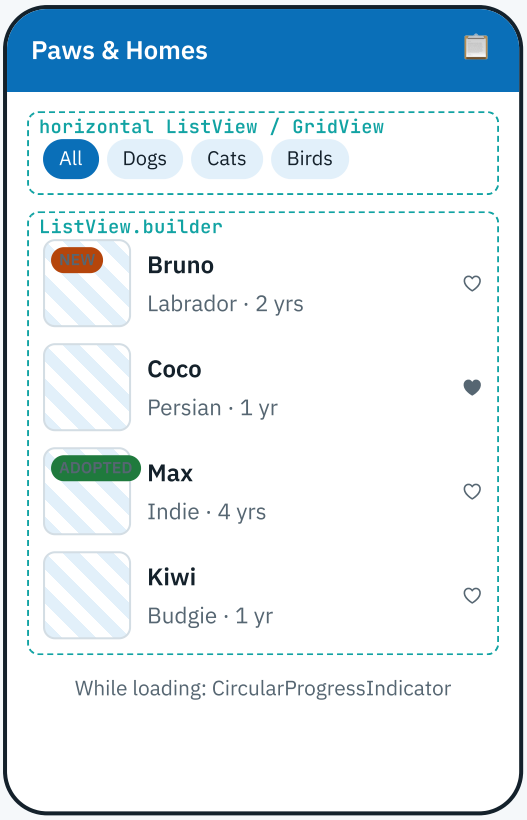
What the app does

- The home screen loads a list of pets (with a loading spinner) and shows them in a scrolling list.
- A row of category chips at the top filters the list: All, Dogs, Cats, Birds.
- Each pet card has a heart button to mark it as a favourite.
- Tapping a pet opens its profile with a large photo, three stat tiles and an **Adopt me** button.
- The adoption form checks every field before it can be submitted.
- After submitting, the pet shows an **Adopted** badge and a SnackBar confirms it.
- An icon in the home AppBar opens **My applications**, a list of everything you've applied for.

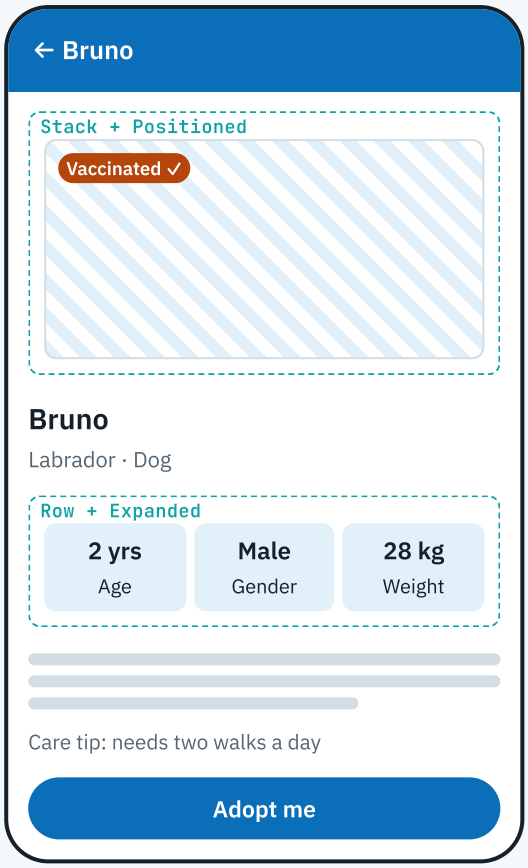
Screens

Dashed boxes show the widget to use for each part. The text above each phone is its route name.

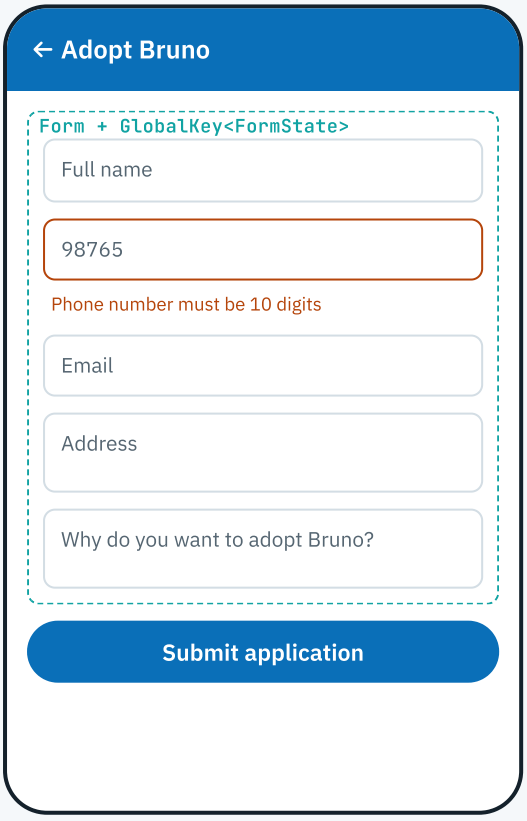
AppRoutes.home · '/'



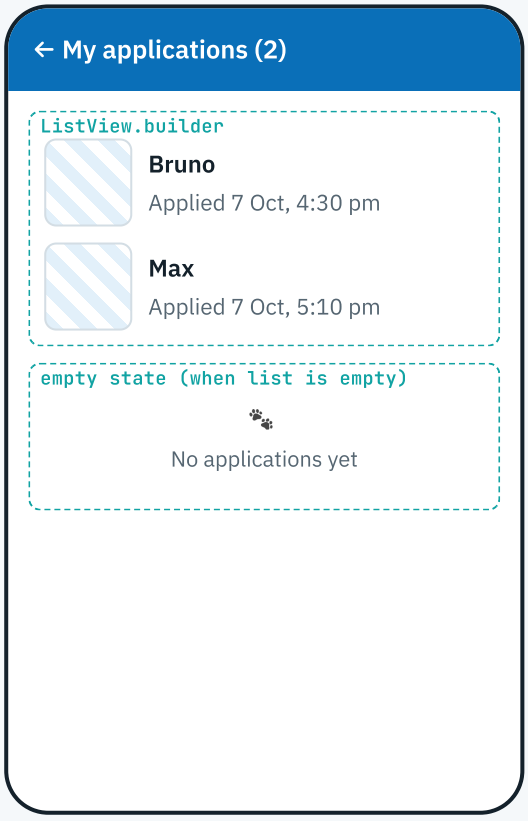
AppRoutes.petDetail · '/pet'



AppRoutes.adopt · '/adopt'



AppRoutes.applications · '/applications'



Rules you must follow

1. **Use named routes only.**
   Put every route name in an `AppRoutes` class as constants. Don't use `MaterialPageRoute` anywhere. Pass data with `arguments`.

2. **Take every color and font from the theme.**
   Define them once in `lib/theme.dart` and read them with `Theme.of(context)`. No `Colors.xxx` or `Color(0x…)` in screens or widgets.

3. **Use a custom font**
   from `assets/fonts/` and at least 6 pet images from `assets/images/`, both registered in `pubspec.yaml`.

4. **Build lists with builders.**
   Use `ListView.builder` or `GridView.builder` for every list of pets or applications.

5. **Validate every form field**
   with the rules in the table below, and submit only when `validate()` returns true.

6. **Dispose every `TextEditingController`**
   in `dispose()`.

7. **Check `mounted` after every `await`**
   before using `context` or calling `setState`.

8. **No overflow errors.**
   The app must run without yellow-and-black stripes on a small phone (360 px wide), including with the keyboard open on the form.

9. **No extra packages.**
   Only `flutter` itself. Riverpod, http and Firebase come in later sessions.

10 **Commit as you go.**
At least 5 commits with clear messages (for example “Add pet list screen”), pushed to GitHub.

Requirements by session

Every session so far is part of this project. Use this table to check you haven't missed anything.

SESSION	WHAT YOUR APP MUST INCLUDE
1 · Dart	An **abstract class** `Pet` with subclasses `Dog`, `Cat` and `Bird` that **override** `careTip()`. At least one **named constructor** (for example `Dog.puppy(...)` with age 0). A **nullable** field `String? adoptedBy` and a getter `bool get isAdopted => adoptedBy != null;`. Use a **List** for pets, a **Set** for favourite pet ids and a **Map** for applications keyed by pet id. An **async** function `Future<List<Pet>> fetchPets()` that waits 2 seconds with `Future.delayed`, called inside **try/catch**.
2 · Widgets	The right choice of `StatelessWidget` or `StatefulWidget` for each screen and widget. Core widgets: Text, Container, Image, Icon, SizedBox, Column, Row.
3 · Layouts	Category chips in a horizontal list. Pet cards with an image, name, breed and age. A `Stack` with `Positioned` for the NEW / ADOPTED badges and the Vaccinated label. Three stat tiles in a `Row` with `Expanded`. Padding and alignment that look consistent. A `ThemeData` with a custom font.
4 · Navigation & forms	Four named routes. The detail screen receives a `Pet`. The form returns an `AdoptionRequest` to the detail screen, which returns it to the home screen. `setState` for the filter, favourites and adopted status. `initState` starts loading the pets. A validated `Form` with controllers.

Data model

Start with these classes in `lib/models/`. You can add fields, but don't remove any.

```
abstract class Pet {
  Pet({
    required this.id,
    required this.name,
    required this.breed,
    required this.ageInYears,
    required this.gender,
    required this.weightKg,
    required this.imagePath,
    required this.description,
    this.isVaccinated = false,
    this.isNew = false,
  });

  final String id;
  final String name;
  final String breed;
  final int ageInYears;
  final String gender;
  final double weightKg;
  final String imagePath;
  final String description;
  final bool isVaccinated;
  final bool isNew;

  String? adoptedBy;                          // null until someone adopts
  bool get isAdopted => adoptedBy != null;

  String get type;                            // 'Dog', 'Cat' or 'Bird'
  String careTip();                           // each subclass overrides this
}

class AdoptionRequest {
  // petId, petName, fullName, phone, email, address, reason, submittedAt
(DateTime)
}
```

Sample pets

Use at least these six. Add your own photos for them in `assets/images/`.

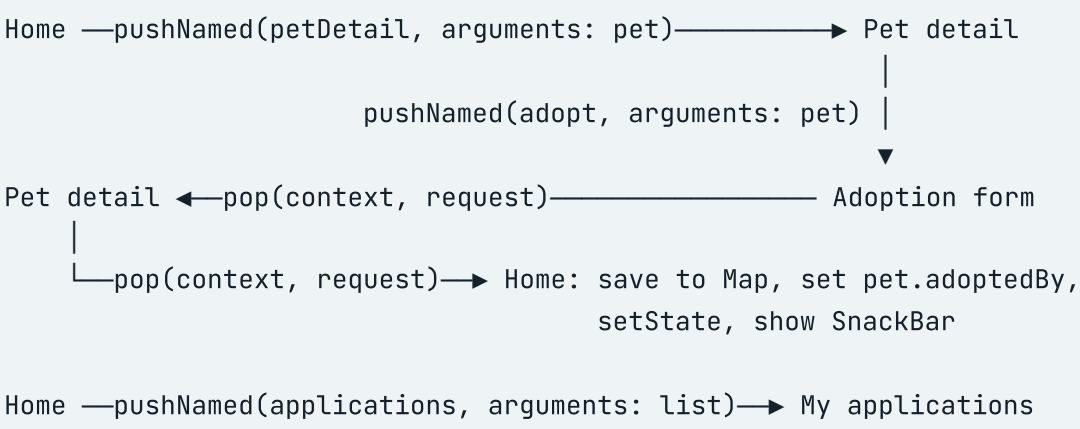
NAME	TYPE	BREED	AGE	GENDER	WEIGHT	NOTES
Bruno	Dog	Labrador	2	Male	28 kg	Vaccinated, new
Coco	Cat	Persian	1	Female	3.5 kg	Vaccinated
Max	Dog	Indie	4	Male	18 kg	Vaccinated
Kiwi	Bird	Budgie	1	Female	0.04 kg	New
Luna	Cat	Siamese	3	Female	4 kg	
Rocky	Dog	Beagle	0	Male	6 kg	Create with `Dog.puppy(...)`

Adoption form rules

FIELD	RULE	ERROR MESSAGE TO SHOW
Full name	Required, at least 3 characters, letters and spaces only	Enter your full name · Use letters only
Phone	Exactly 10 digits, starting with 6, 7, 8 or 9. Use the phone keyboard.	Enter a valid 10-digit mobile number
Email	Required, valid format. Use the email keyboard.	Enter a valid email, like name@example.com
Address	Required, at least 10 characters, 3 lines tall (`maxLines: 3`)	Enter your full address
Reason	Required, at least 20 characters. The label includes the pet's name.	Tell us a little more (at least 20 characters)

Put the validators in `lib/utils/validators.dart` as plain functions, not inside the widget.

How the screens connect



- If the pet is already adopted, the detail screen shows “Already adopted” and disables the button.
- If the user presses back on the form, nothing changes. Handle the `null` result.

Folder structure

```
lib/
├─ main.dart · MaterialApp, theme, routes map
├─ theme.dart
├─ routes.dart · class AppRoutes
├─ models/ · pet.dart, adoption_request.dart
├─ data/ · pet_service.dart with fetchPets()
├─ utils/ · validators.dart
├─ screens/ · one file per screen
└─ widgets/ · pet_card.dart, category_chips.dart, stat_tile.dart
```

Grading

AREA	WHAT WE CHECK	POINTS
Dart & models	Abstract class, overrides, named constructor, null safety, List / Set / Map, async with try/catch	15
Layout & styling	Matches the wireframes, Stack badges, stat tiles, theme only, custom font, no overflow	20
Navigation	Named routes with constants, data passed forward, results returned through two screens	20
Forms	All five rules, clear error messages, correct keyboards, controllers disposed	20
State & lifecycle	Loading spinner from `initState`, filter, favourites and adopted status update correctly, `mounted` checks	10
Code quality & Git	Folder structure, reusable widgets, 5+ clear commits, README with screenshots	15
Total		100

Bonus challenges

- [ ] Add a search `TextField` above the list that filters pets by name as you type.
- [ ] Make `fetchPets()` fail randomly once in three tries, and show an error message with a **Retry** button.
- [ ] On My applications, let the user withdraw an application. The pet becomes available again on the home screen.
- [ ] Add a “Favourites only” chip to the category row.

How to submit

- [ ] Push your project to a public GitHub repository.
- [ ] Add a README with the app name, how to run it, and screenshots of all four screens plus one screenshot showing a form error.
- [ ] Run `flutter analyze` and fix every warning before you submit.

- [ ] Send the repository link to your teacher.